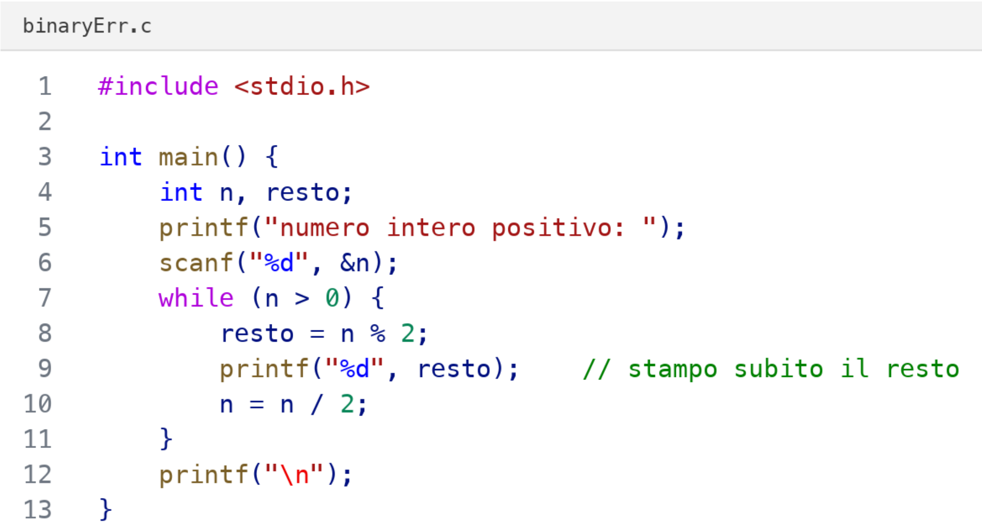
Array Statici in C

Conversione da base 10 → base 2

n	n/2	resto
13	6	1_{1}
6	3	0_{2}
3	1	1_{3}
1	0	1_{4}

numero giusto
1101


binaryErr.c

```c
#include <stdio.h>

int main() {
    int n, resto;
    printf("numero intero positivo: ");
    scanf("%d", &n);
    while (n > 0) {
        resto = n % 2;
        printf("%d", resto);     // stampo subito il resto
        n = n / 2;
    }
    printf("\n");
}
```

appare 1011 non va bene

Soluzione: uso di array

Senza array int cifra1, cifra2, cifra3, - -

```c
int cifra[8]; // array statico di
              lunghezza 8
```

un int occupa 32 bit (4 byte)

cifra rapp 8 variabili di tipo int
=> 8 x 4 = 32 byte

int n;

n
3
32 bit
4 byte

n = 3

int cifra[8];

cifra[7]
cifra[0]
cifra[1]
cifra[2]
cifra[3]
cifra[4]
cifra[5]
-3
-1
51
4 byte
32 byte

cifra[0] = -3

cifra[5] = 51

cifra[4] = -1.34501

tutti elementi di array sono dello stesso tipo

cifra[8] = 32;

Errore index out of boun

array lungo N.

indici 0, 1, --, N-1

Dichiarazione `double dati[1000];`

Errore Comune:

```
printf("inserisci lunghezza array: ");
int n;
scanf("%d", &n);
double dati[n];
```

n deve essere noto al tempo di compilaz.

non va bene

Soluzione con array statici

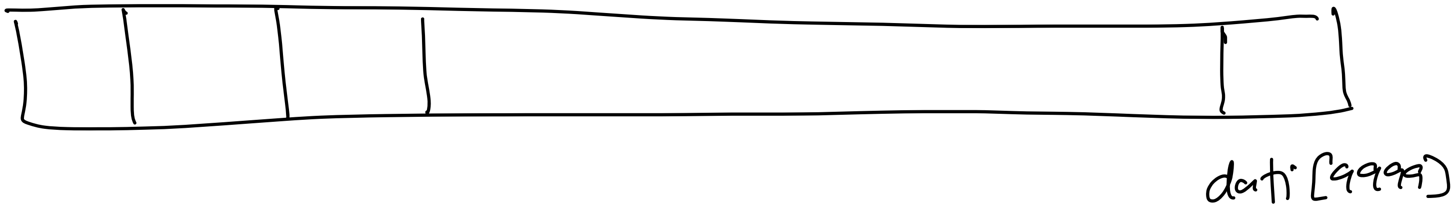
```
#define NMAX 10000

int main() {
    double dati[NMAX];
    printf(" numero di esperimenti < %d:", NMAX);
    int n;
    scanf("%d", &n);
    int i;
    for(i=0; i<n; i++) {
        dati[i] = -1 + 2*drand48();
    }
```

$X \in [-1,1)$

drand48(): $\in [0,1)$

2*drand48(): $\in [0,2)$

double dati[10000]



n = 3

⇒ dati[0] = - 0.953

dati[1] = 0.213

dati[2] = 9.95

dati[3] = ? spazzatura

⇒ Sempre inizializzare l'array prima dell'uso

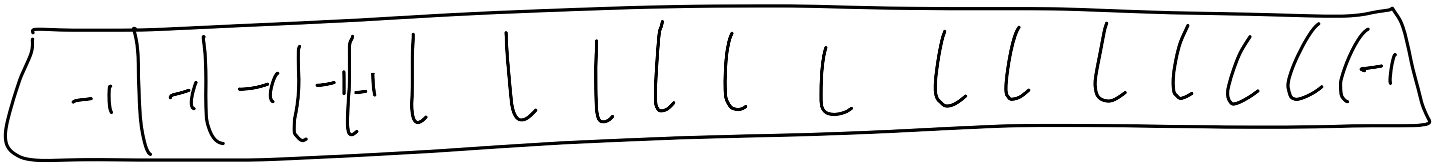
int ~~dati[10000];~~

~~dati = 0;~~

[ ] : operatore di accesso all'elem. array

int ~~dati[1000]~~ = 0;

int dati[1000] = {-1};



int voti[4] = {30, 28, 29, 29};

tanti valori quanto la lunghezza

v

int dati[1000] = {0}; ✓

dati[123] = -1;

dati[12] = 13;

~~dati[1000] = 2;~~ ✗ errore out of bound

dati[999] = 25;

Probleme delle Conversione:

8 cifre => int cifre[8];

numero più grande: $2^8 - 1 = 255$

#define NCIFRE 8

int main() {
int cifre [NCIFRE]

do {
printf("inserisci 0<=n<=%d:", ~~pow(2,8)-1~~);
pow(2,NCIFRE)-1);
} while(n<0 || n>=pow(2,NCIFRE));

n	n/2	n%2
13	6	1
6	3	0
3	1	1
1	0	1

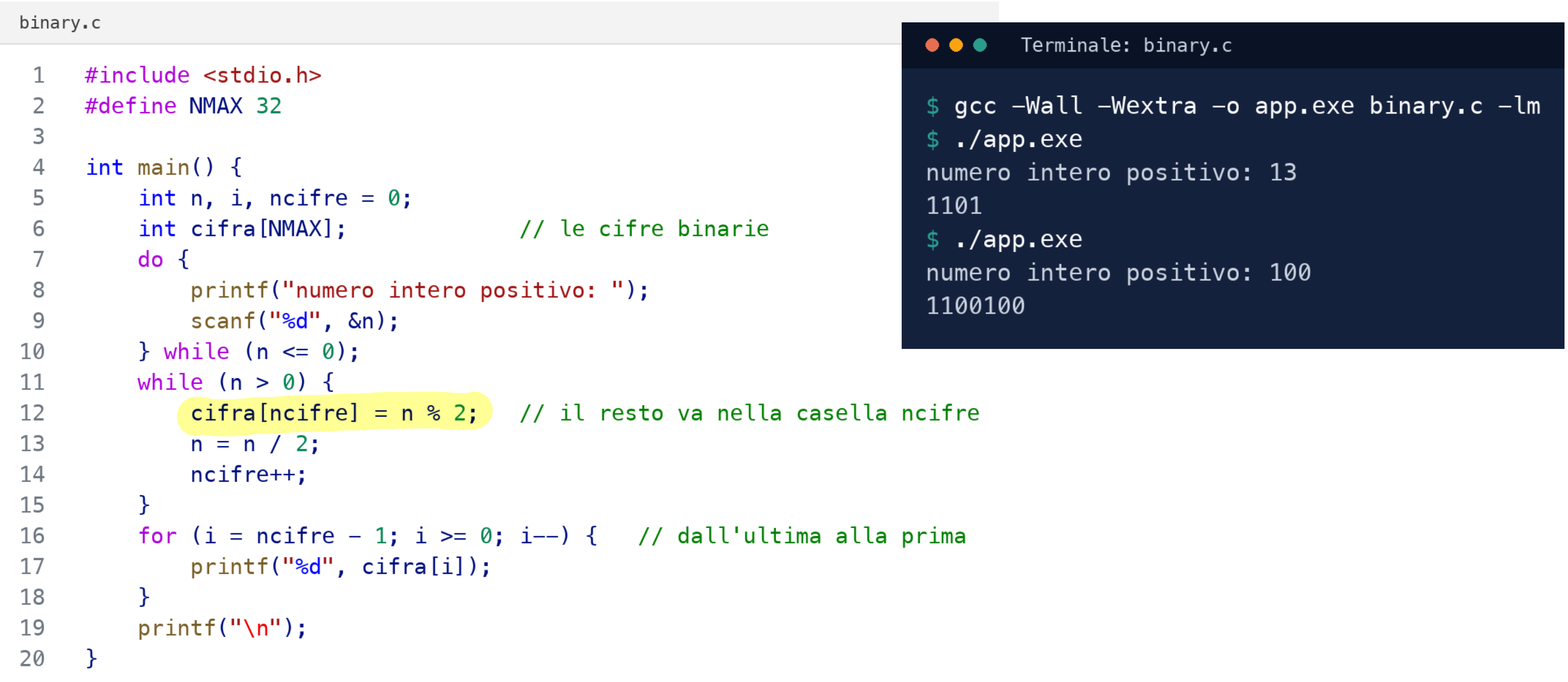
```
int n = 13;
    int icifra = 0;
    while ( n>0 ) {
        resto = n % 2;
        cifra [icifre] = n%2;
        n = n/2;
        icifre ++;
    }
```

i	cifre[i]
0	1
1	0
2	1
3	1

alla fine del ciclo icifre = 3

```
for (int i = icifra-1; i >=0 ; i-- ) {
    printf(" %d ", cifre[i] ;
}
printf("\n");
```

cifra[3]	cifre[2]	cifre[1]	cifre[0]
1	1	0	1



```c
#include <stdio.h>
#define NMAX 32

int main() {
    int n, i, ncifre = 0;
    int cifra[NMAX];              // le cifre binarie
    do {
        printf("numero intero positivo: ");
        scanf("%d", &n);
    } while (n <= 0);
    while (n > 0) {
        cifra[ncifre] = n % 2;    // il resto va nella casella ncifre
        n = n / 2;
        ncifre++;
    }
    for (i = ncifre - 1; i >= 0; i--) {    // dall'ultima alla prima
        printf("%d", cifra[i]);
    }
    printf("\n");
}
```

```
$ gcc -Wall -Wextra -o app.exe binary.c -lm
$ ./app.exe
numero intero positivo: 13
1101
$ ./app.exe
numero intero positivo: 100
1100100
```